

Mathematics for Systems Research and Engineering

A structured curriculum in proof, operators, uncertainty, dynamics, and computation

A competency-based route from mathematical intuition to explicit assumptions, reproducible computation, and reviewable system claims.

Proof and mathematical language

Operators and representation

Uncertainty and information

Dynamics and computation

Publication note

This guide presents a selective independent-study curriculum for people who design, analyse, or re-search technical systems. It is intended for engineers, scientific programmers, AI and machine-learning practitioners, control-system designers, independent researchers, and technically ambitious self-learners who need more than a survey of mathematical topics but do not require a conventional degree sequence.

The curriculum is organised around five recurring capabilities:

1. **Proof:** state claims precisely and determine what has actually been established.
2. **Operators and representation:** describe transformations, state spaces, invariants, and modes of behaviour.
3. **Uncertainty:** reason about evidence, inference, information, and stochastic variation without relying on vague language.
4. **Dynamics and control:** model how systems evolve, identify stable and unstable regimes, and design feedback.
5. **Computation:** understand approximation, conditioning, numerical error, and optimisation well enough to turn theory into reliable software.

This public edition generalises an earlier internal curriculum. Project-specific terminology has been removed, mathematical statements have been tightened, and the resource sequence has been revised for independent use.

Scope and limitations. This is an independent-study framework, not an accredited programme and not a substitute for formal instruction where professional registration, regulated practice, or academic credit is required. It is deliberately selective rather than exhaustive. Resource availability, course numbering, and external licensing can change; readers should confirm the current terms of each external source.

Copyright. Copyright (c) 2026 Thomas Helm / ToraFirma Systems. All rights reserved unless a separate repository notice states otherwise. External resources remain subject to their own terms and licences. Inclusion does not imply affiliation or endorsement.

Contents

1. Purpose, audience, and outcomes
2. Curriculum architecture and entry routes
3. Phase 0 - Proof and mathematical language
4. Phase 1 - Linear algebra and the operator viewpoint
5. Phase 2 - Real analysis
6. Phase 3 - Probability, statistics, and information theory
7. Phase 4 - Differential equations, dynamical systems, and control
8. Phase 5 - Structural mathematics: algebra, geometry, and complex analysis
9. Phase 6 - Functional analysis, numerical analysis, and optimisation
10. Synthesis practicum: translating mathematics into systems work
11. Study cadence and evidence of mastery
12. Suggested pathways and emphasis order
13. Mastery checkpoints
14. Resource policy
15. Resource directory
16. Working templates
17. Final operating principle

1. Purpose, audience, and outcomes

1.1 Why this curriculum exists

Technical work often fails at the boundary between intuition and formal structure. A designer may recognise a pattern but be unable to state the relevant object. An engineer may run a successful simulation without knowing whether the result is robust. A researcher may invoke stability, entropy, curvature, symmetry, or optimality while leaving the assumptions implicit. In each case, the central problem is not a lack of ideas. It is a missing chain from idea to definition, from definition to theorem, from theorem to computation, and from computation to falsification.

This curriculum is designed to build that chain.

It does not attempt to cover every branch of mathematics. Instead, it develops a coherent spine for systems research and engineering: proof, linear structure, convergence, uncertainty, dynamics, advanced structure, and computation. The sequence is chosen so that each phase improves both mathematical judgement and practical system design.

1.2 Who it is for

The guide is suitable for readers who already work with one or more of the following:

- software architecture, algorithms, or formal specifications;
- machine learning, statistical modelling, or data systems;
- signal processing, control, robotics, or simulation;
- physics-informed computation or scientific programming;
- complex systems, dynamical models, or optimisation;
- independent mathematical or interdisciplinary research.

A reader does not need a university mathematics degree to begin. They do need persistence, comfort with symbolic notation, and a willingness to solve problems rather than only read explanations.

1.3 What this guide is not

This is not:

- a list of interesting books;
- a promise of rapid mastery;
- a replacement for exercises, proofs, or computation;
- a licence to use mathematical language without checking hypotheses;
- a fixed calendar that treats completion as equivalent to competence.

The curriculum is **gate-based**. Progress is demonstrated through written proofs, derivations, numerical experiments, counterexamples, and system translations. The relevant question is not “Have I watched the lectures?” but “Can I produce evidence that I can use the mathematics correctly?”

1.4 Intended outcomes

A reader who completes the core spine and passes the mastery gates should be able to:

- define mathematical objects and domains before making claims about them;
- distinguish analogy, approximation, equivalence, and isomorphism;
- move between coordinate-free objects and matrix or computational representations;
- test convergence, stability, identifiability, and optimisation claims against their assumptions;
- formulate uncertainty using probability distributions, estimators, entropy, divergence, and information;
- analyse linear and nonlinear dynamics using phase portraits, spectra, Lyapunov functions, and feedback;
- assess numerical conditioning, stability, approximation error, and floating-point effects;
- translate a system claim into objects, assumptions, invariants, observables, experiments, and failure modes;
- communicate results in a form that another person can independently inspect or reproduce.

1.5 Minimum preparation

The core sequence assumes:

- algebraic manipulation, functions, graphs, and basic trigonometry;
- familiarity with derivatives and integrals, even if review is required;
- basic programming ability for numerical experiments;
- willingness to learn proof notation and mathematical writing.

Readers with weak calculus should use OpenStax Calculus during Phase 0. Readers with solid calculus but limited proof experience should still complete the Phase 0 diagnostic and proof gate.

2. Curriculum architecture and entry routes

2.1 Phase map

Phase	Focus	Principal capability
0	Proof, discrete mathematics, and calculus repair	Exact definitions, logical structure, invariants, and disciplined symbolic work
1	Linear algebra and operators	Representation, subspaces, spectra, approximation, and linear state evolution
2	Real analysis	Convergence, continuity, compactness, and rigorous limit arguments
3	Probability, statistics, and information theory	Uncertainty, evidence, inference, entropy, and dependence
4	Differential equations, dynamical systems, and control	Evolution, regimes, stability, bifurcation, feedback, and state estimation
5	Algebra, geometry, and complex analysis	Symmetry, equivalence, curvature, analytic structure, and transforms
6	Functional analysis, numerical analysis, and optimisation	Infinite-dimensional operators, reliable computation, and tractable decision procedures
S	Synthesis practicum, run throughout	Translation from theorem to model, implementation, evidence, and failure analysis

2.2 Sequence rules

Phases 0 through 4 form the core spine. They should normally be studied in order because each one supplies language used by the next. Phase 5 and Phase 6 contain advanced modules that can be interleaved once the corresponding prerequisites are stable.

The synthesis practicum is not an end-of-course capstone. It begins in Phase 0 and continues throughout. Every major concept should eventually be connected to a concrete model, algorithm, experiment, or design decision.

The sequence follows four rules:

- 1. Definitions before slogans.** Learn the exact object before relying on an intuitive summary.
- 2. Examples and counterexamples together.** A theorem is understood only when its boundary is visible.
- 3. Proof and computation together.** Proof controls generality; computation exposes implementation and numerical behaviour.

- 4. Mastery gates before expansion.** Do not compensate for a weak foundation by collecting more advanced terminology.

2.3 Entry routes

Reader profile	Recommended starting route
Proof experience is limited or inconsistent	Begin with Phase 0 in full, then proceed sequentially
Proof skills are sound but linear algebra is procedural	Complete the Phase 0 diagnostic, then begin Phase 1
Working ML or data practitioner	Phase 0 diagnostic -> Phase 1 -> Phase 3 -> Phase 2 -> Phase 4 -> Phase 6
Control, robotics, or simulation practitioner	Phase 0 diagnostic -> Phase 1 -> Phase 2 -> Phase 4 -> Phase 6; add Phase 3 early if systems are stochastic
Theoretical or interdisciplinary researcher	Follow the complete sequence and begin the synthesis practicum immediately
Strong undergraduate mathematics background	Use the mastery gates as diagnostics, skip only material that can be demonstrated in writing, and concentrate on synthesis and numerical verification

2.4 Diagnostic rule

Skipping a phase should be based on evidence, not familiarity. A reader may test out of a phase only by completing its mastery gate without notes and producing a short written artifact that another technically competent reader could review.

3. Phase 0 - Proof and mathematical language

3.1 Purpose

Phase 0 establishes the discipline required by every later phase. The aim is not to make elementary material ceremonial. It is to eliminate hidden ambiguity in definitions, quantifiers, implications, induction, and symbolic manipulation before those weaknesses are amplified by advanced topics.

Proof is also an engineering skill. An invariant proof, a termination argument, a precondition/postcondition contract, and a safety argument all depend on the same ability to state a domain, identify assumptions, and preserve a logical chain.

3.2 Core topics

- propositions, predicates, quantifiers, implication, equivalence, and negation;
- sets, functions, relations, injections, surjections, and bijections;
- direct proof, contrapositive, contradiction, induction, and minimal-counterexample arguments;
- counting, recurrence relations, graphs, state machines, and elementary discrete probability;
- invariants and monovariants in discrete processes;
- asymptotic notation and basic algorithmic reasoning;
- calculus repair as needed: limits, derivatives, integrals, and the fundamental theorem of calculus;
- mathematical writing: definitions, theorem statements, proof structure, and explicit conclusions.

3.3 Working standard

A correct proof should make the following visible:

- the domain of each object;
- the assumptions being used;
- the claim to be established;
- the logical method;
- the step where each assumption is applied;
- the conclusion and its scope.

Avoid phrases such as “clearly”, “obviously”, or “it follows” when they conceal the only non-trivial step. Concision is valuable only after the argument is complete.

3.4 Mastery gate

Produce a short portfolio containing:

1. a proof by induction with a correct base case, inductive hypothesis, and inductive step;
2. a proof by contradiction for a basic number-theoretic or combinatorial claim;
3. a quantifier-negation exercise covering nested universal and existential quantifiers;

4. an invariant proof for a small state machine or iterative algorithm;
5. a solved linear recurrence with initial conditions and a verified closed form.

The portfolio passes only when every symbol is defined and no logical step depends on an unstated assumption.

3.5 Practice artifacts

- **Proof notebook:** one complete proof per study session.
- **Definitions ledger:** exact definitions, examples, non-examples, and dependencies.
- **Error log:** the failed step, the reason it failed, and the corrected argument.
- **Invariant sheet:** state, transition, invariant, initialisation proof, preservation proof, and consequence.

3.6 Primary resources

Role	Resource	Use
Primary	MIT 6.042J, Mathematics for Computer Science	Proof methods, discrete structures, state machines, counting, recurrences, and discrete probability
Repair/reference	OpenStax Calculus, Volume 1	Review of functions, limits, derivatives, integrals, and single-variable calculus

4. Phase 1 - Linear algebra and the operator viewpoint

4.1 Purpose

Linear algebra provides the main representational language of modern engineering, scientific computing, machine learning, control, and signal processing. The essential shift is from treating matrices as rectangular arrays to treating them as representations of linear maps between vector spaces.

This phase develops the ability to ask:

- What information does a transformation preserve or destroy?
- Which states are reachable, observable, or indistinguishable?
- Which directions grow, decay, rotate, or remain invariant?
- What is the best lower-dimensional approximation?
- How sensitive is a solution to perturbation?

4.2 Core topics

Vector spaces and linear maps

- span, linear independence, basis, dimension, coordinates;
- kernel, image, rank, and nullity;
- the four fundamental subspaces of a matrix;
- change of basis and similarity;
- coordinate-free objects versus matrix representations.

Orthogonality and approximation

- inner products, norms, orthogonal complements, and Gram-Schmidt;
- orthogonal projections and best approximation in a subspace;
- least squares, normal equations, QR factorisation, and pseudoinverses;
- why QR or SVD is generally preferred to normal equations in numerical work: forming the normal equations squares the condition number.

Spectral structure

- eigenvalues, eigenvectors, invariant subspaces, characteristic and minimal polynomials;
- diagonalisation and Jordan structure;
- symmetric, Hermitian, normal, orthogonal, and unitary operators;
- spectral theorem and positive-definite matrices;
- quadratic forms and Rayleigh quotients.

Singular value decomposition

- SVD as orthogonal change of coordinates, non-negative axis scaling, and a second orthogonal change of coordinates;
- rank, conditioning, pseudoinverse, principal directions, and low-rank approximation;
- Eckart-Young-Mirsky theorem and approximation error.

Linear systems and state-space models

- continuous systems $\dot{x}(t) = A x(t)$ and matrix exponentials;
- discrete systems $x[k+1] = A x[k]$;
- modal decomposition and transient behaviour;
- the origin of a finite-dimensional discrete linear system is asymptotically stable exactly when the spectral radius $\rho(A) < 1$;
- controllability and observability matrices, reachable subspaces, observable subspaces, and Gramians under appropriate stability assumptions.

4.3 Mathematical cautions

- Eigenvalues alone do not describe every transient phenomenon. Non-normal matrices can exhibit large transient amplification even when all eigenvalues indicate asymptotic decay.
- A projection satisfies $P^2 = P$; the norm-minimising property requires an **orthogonal** projection, equivalently $P^* = P$ in an inner-product space.
- A matrix representation depends on a basis. The underlying linear map does not.
- Diagonalisation is not guaranteed. Jordan form is mathematically useful but can be numerically ill-conditioned.

4.4 Mastery gate

Produce a worked analysis of a non-trivial matrix or linear operator that includes:

1. rank, kernel, image, and the four fundamental subspaces;
2. an orthogonal projection and a least-squares solution;
3. QR and SVD interpretations of the same approximation problem;
4. eigenstructure, diagonalisation status, and a stability analysis of $x[k+1] = A x[k]$;
5. a short explanation of what is invariant under a change of basis;
6. a numerical conditioning check using a small perturbation experiment.

4.5 Practice artifacts

- operator profile: domain, codomain, kernel, image, adjoint, spectrum, norm, and invariants;
- low-rank approximation notebook with error curves;
- transient-growth comparison for a normal and a non-normal matrix;
- controllability and observability analysis for a small state-space model.

4.6 Primary resources

Role	Resource	Use
Primary	MIT 18.06SC, Linear Algebra	Independent-study sequence covering systems, subspaces, least squares, eigenvalues, SVD, and positive-definite matrices
Applied companion	Stanford EE263, Introduction to Linear Dynamical Systems	Applied linear algebra for signals, estimation, control, and state-space systems
Advanced companion	MIT 18.S096, Matrix Calculus for Machine Learning and Beyond	Derivatives as linear maps, Jacobians, matrix-valued differentiation, and operator-aware calculus

5. Phase 2 - Real analysis

5.1 Purpose

Real analysis turns informal calculus into a reliable theory of limits, approximation, continuity, and convergence. It is the phase where statements such as “the approximation converges”, “the limit can be moved inside”, or “the solution depends continuously on the input” acquire exact hypotheses.

The practical purpose is to stop treating smoothness and convergence as visual impressions. The reader should learn to recognise the compactness, boundedness, completeness, uniformity, and domination assumptions that make limit arguments valid.

5.2 Core topics

- real-number completeness, suprema and infima, and the Archimedean property;
- sequences, subsequences, series, Cauchy sequences, and completeness;
- epsilon-delta definitions of limits and continuity;
- differentiability, mean value theorem, Taylor’s theorem, and remainder estimates;
- pointwise versus uniform convergence;
- exchanging limits with integration or differentiation;
- Riemann integration and the fundamental theorem of calculus;
- compactness, Heine-Borel, extreme values, and uniform continuity;
- metric spaces and contraction mappings;
- Banach fixed-point theorem on a complete metric space;
- counterexamples that separate continuity, uniform continuity, differentiability, and integrability.

5.3 Working standard

For every convergence or limit claim, identify:

1. the space in which convergence occurs;
2. the mode of convergence;
3. the norm, metric, or topology being used;
4. the theorem that permits the desired operation;
5. the hypothesis that would fail in a counterexample.

This discipline becomes essential later when dealing with stochastic convergence, numerical convergence, operator limits, and optimisation algorithms.

5.4 Mastery gate

Produce a portfolio containing:

1. an epsilon-N proof of sequence convergence;

2. an epsilon-delta proof of continuity;
3. a concrete sequence of functions that converges pointwise but not uniformly;
4. a proof that a continuous function on a compact interval is uniformly continuous;
5. a correct statement and application of the Banach fixed-point theorem, including completeness and the contraction constant;
6. a counterexample showing why a tempting interchange of limits is invalid without an additional hypothesis.

5.5 Practice artifacts

- convergence taxonomy: pointwise, uniform, almost sure, in probability, in distribution, and in norm, updated as later phases are studied;
- theorem dependency map showing where compactness or completeness enters;
- “hidden assumption” memos for proofs found in engineering or ML papers;
- fixed-point iteration notebook comparing theory with observed convergence rates.

5.6 Primary resources

Role	Resource	Use
Primary	MIT 18.100B, Real Analysis	Proof-centred introduction to real numbers, convergence, continuity, metric spaces, differentiation, and integration
Companion	Tao, Analysis I and II	Detailed construction and pedagogical development of analysis
Reference	Rudin, Principles of Mathematical Analysis	Dense theorem-and-exercise reference after the primary course is stable

6. Phase 3 - Probability, statistics, and information theory

6.1 Purpose

This phase replaces vague references to uncertainty, evidence, confidence, entropy, and signal quality with explicit probabilistic models and inferential procedures. It also develops the judgement needed to separate randomness in the world from uncertainty in a model, estimator, or decision process.

Probability provides the model. Statistics provides procedures for learning from observations. Information theory quantifies limits on representation, dependence, compression, and communication.

6.2 Core topics

Probability foundations

- sample spaces, events, sigma-algebras, and probability measures;
- discrete and continuous random variables;
- cumulative distribution functions, probability mass functions, and densities;
- expectation, moments, variance, covariance, and conditional expectation;
- independence, conditional independence, and Bayes' rule;
- joint, marginal, and conditional distributions;
- transformations of random variables.

Concentration and asymptotics

- Markov, Chebyshev, Hoeffding, and Chernoff-type bounds;
- laws of large numbers;
- central limit theorem;
- convergence almost surely, in probability, in distribution, and in mean square;
- when asymptotic approximations are inappropriate for finite samples.

Statistical inference

- likelihood and maximum-likelihood estimation;
- estimators, bias, variance, consistency, and efficiency;
- confidence intervals and hypothesis tests;
- Bayesian inference, priors, posteriors, and posterior predictive distributions;
- regression and model checking;
- identifiability, confounding, multiple comparisons, and calibration.

Information theory

- Shannon entropy, conditional entropy, cross-entropy, and KL divergence;
- mutual information and data-processing inequalities;

- entropy rates for stochastic processes;
- lossless and lossy source coding;
- channel capacity and coding theorems;
- variational characterisations where appropriate.

6.3 Mathematical cautions

- Independence implies zero covariance when moments exist; zero covariance does not generally imply independence.
- Shannon entropy is maximised by the uniform distribution only when the finite support is fixed.
- Differential entropy can be negative and is not invariant under a change of coordinates; KL divergence and mutual information have different invariance properties.
- A p-value is not the probability that a hypothesis is true.
- A confidence interval and a Bayesian credible interval answer different questions.
- High predictive accuracy does not establish causal validity, calibration, or robustness under distribution shift.

6.4 Mastery gate

Complete a small probabilistic study that includes:

1. a fully specified probability model and its assumptions;
2. calculation of expectation, variance, covariance, and a conditional distribution;
3. a sequential Bayesian update;
4. a frequentist estimate with an uncertainty interval and an explanation of its interpretation;
5. calculation of $H(X)$, $H(X|Y)$, $I(X;Y)$, and $D(P || Q)$ for a non-trivial discrete example;
6. a simulation that compares an asymptotic approximation with finite-sample behaviour;
7. a short section on identifiability, calibration, and possible distribution shift.

6.5 Practice artifacts

- uncertainty register: aleatoric uncertainty, epistemic uncertainty, model-form uncertainty, and measurement error;
- Bayesian update notebook;
- concentration-bound comparison against Monte Carlo results;
- entropy and mutual-information calculator for small distributions;
- model-card appendix describing assumptions and failure conditions.

6.6 Primary resources

Role	Resource	Use
Primary	MIT 6.041SC, Probabilistic Systems Analysis and Applied Probability	Independent-study treatment of probability models, random variables, random processes, laws of large numbers, and inference
Primary advanced	MIT 6.441, Information Theory	Entropy, divergence, hypothesis testing, compression, channel coding, and modern information theory
Practical companion	OpenStax, Introductory Statistics 2e	Accessible reference for estimation, testing, regression, and statistical practice
Reference	Cover and Thomas, Elements of Information Theory	Standard graduate reference after probability foundations are secure

7. Phase 4 - Differential equations, dynamical systems, and control

7.1 Purpose

This phase develops a formal language for evolution over time. It connects state, transition laws, equilibria, oscillations, attractors, bifurcations, feedback, and estimation. It is central to physical systems, control, optimisation dynamics, training processes, iterative algorithms, and any architecture whose behaviour depends on its history.

The key conceptual shift is from asking only what a system computes to asking how its state moves, what regimes exist, which perturbations decay, and what intervention changes the long-term behaviour.

7.2 Core topics

Differential equations

- first- and second-order ordinary differential equations;
- separation, integrating factors, characteristic equations, and variation of parameters;
- existence and uniqueness of solutions;
- linear systems, matrix exponentials, and fundamental matrices;
- forced response, resonance, Laplace transforms, and frequency-domain interpretation;
- numerical integration and local/global truncation error.

Qualitative dynamics

- autonomous systems, flows, equilibria, invariant sets, and trajectories;
- phase lines, nullclines, and phase portraits;
- stable and unstable manifolds;
- limit cycles, Poincaré maps, and recurrence;
- saddle-node, transcritical, pitchfork, and Hopf bifurcations;
- chaos, sensitivity, and the limits of local analysis.

Stability

- Lyapunov, asymptotic, exponential, and input-to-state stability;
- linearisation near equilibria;
- reliable conclusions for hyperbolic equilibria and the inconclusive nature of non-hyperbolic linearisation;
- Lyapunov functions, LaSalle's invariance principle, and regions of attraction;
- continuous and discrete Lyapunov equations;
- robustness margins and perturbation analysis.

Control and estimation

- state-space models, inputs, outputs, and feedback;
- controllability, stabilisability, observability, and detectability;
- pole placement and observer design;
- linear quadratic regulator and Riccati equations;
- Kalman filtering as state estimation under a probabilistic model;
- model-predictive control and constraints;
- robust and stochastic control;
- underactuated systems and the use of natural dynamics.

7.3 Mathematical cautions

- Local stability does not imply global stability.
- A decreasing scalar diagnostic is not automatically a Lyapunov function. Positivity, domain, derivative or difference conditions, and the relevant equilibrium must be established.
- Eigenvalues of a Jacobian describe local linearised behaviour; they do not by themselves certify nonlinear behaviour far from the equilibrium.
- Controllability and observability are model-relative properties. Discretisation, unmodelled dynamics, actuator limits, and sensor noise can alter practical conclusions.
- Numerical integration can introduce or suppress apparent stability. Step size and method stability regions matter.

7.4 Mastery gate

Produce a complete analysis of a two-dimensional nonlinear system:

1. derive or state the governing equations and domain;
2. find equilibria and compute the Jacobian;
3. classify hyperbolic equilibria and explain any non-hyperbolic cases;
4. draw and numerically verify a phase portrait;
5. identify an invariant set or construct a Lyapunov argument where possible;
6. vary one parameter and document a qualitative regime change;
7. add a control input, state the control objective, and analyse controllability or stabilisability;
8. compare two numerical integration methods and discuss error or stability.

7.5 Practice artifacts

- phase-portrait atlas;
- local-versus-global stability memo;
- Lyapunov certificate with explicit assumptions;
- bifurcation notebook;
- LQR and observer implementation for a small system;
- discretisation study showing how step size changes numerical behaviour.

7.6 Primary resources

Role	Resource	Use
Primary	MIT 18.03, Differential Equations	ODE methods, linear systems, transforms, resonance, and qualitative interpretation
Primary advanced	MIT Underactuated Robotics	Nonlinear dynamics, Lyapunov analysis, optimal and robust control, planning, estimation, and underactuated systems
Applied companion	Stanford EE263	Linear dynamical systems, estimation, and control through applied linear algebra
Intuition companion	Strogatz, Nonlinear Dynamics and Chaos	Geometric development of phase portraits, bifurcations, oscillations, and chaos

8. Phase 5 - Structural mathematics

Phase 5 develops deeper languages for symmetry, equivalence, geometry, and analytic structure. Its three modules can be interleaved, but each has prerequisites. Abstract algebra depends on proof maturity. Differential geometry depends on multivariable calculus, linear algebra, and analysis. Complex analysis depends on calculus and differential equations.

8.1 Abstract algebra

Purpose

Abstract algebra studies operations and structure independently of a particular coordinate representation. It provides exact tools for distinguishing systems that are merely similar from systems connected by a structure-preserving map.

Core topics

- groups, subgroups, cyclic groups, cosets, and quotient groups;
- homomorphisms, kernels, images, and isomorphism theorems;
- group actions, orbits, stabilisers, and symmetry;
- permutation, matrix, rotation, and finite groups;
- rings, ideals, quotient rings, polynomial rings, and fields;
- modules and linear representations where useful;
- introductory representation theory and character intuition;
- universal properties and commuting diagrams as disciplined structural language.

Mastery gate

Analyse one concrete symmetry or composition system by:

1. defining the set and operation;
2. proving the required algebraic axioms;
3. identifying substructures, homomorphisms, kernels, and quotients;
4. constructing or rejecting a proposed isomorphism;
5. representing the action linearly where appropriate;
6. explaining which properties are invariant under the chosen maps.

Primary resource

MIT 18.701, Algebra I, supplemented by a standard algebra text such as Artin or Dummit and Foote.

8.2 Differential geometry

Purpose

Differential geometry provides a rigorous language for curved spaces, constraints, local coordinates, metrics, geodesics, and curvature. It is relevant whenever states or parameters live on a manifold rather than an unconstrained Euclidean space.

Core topics

- parametrised curves and surfaces;
- tangent vectors, tangent spaces, normals, and coordinate charts;
- arc length and the Frenet-Serret frame;
- curvature of a curve, $\kappa = ||dT/ds||$;
- first and second fundamental forms;
- principal curvatures, Gaussian curvature $K = \kappa_1 \kappa_2$, and mean curvature;
- geodesics, metrics, and shortest paths;
- local versus global structure;
- differential forms and integration on manifolds;
- Gauss-Bonnet as a model of local-to-global reasoning.

Mathematical cautions

- Curvature is local. Locally small curvature does not imply global convexity or simple topology.
- A coordinate singularity is not necessarily a geometric singularity.
- Euclidean gradients and distances are generally incorrect when the state is constrained to a manifold.

Mastery gate

Choose a curve or surface and provide:

1. a parametrisation and valid coordinate domain;
2. tangent and normal fields;
3. arc length or metric calculation;
4. curvature calculation;
5. at least one geodesic or variational argument;
6. a coordinate-change check separating geometric quantities from coordinate artefacts.

Primary resource

MIT 18.950, Differential Geometry, supplemented by do Carmo's *Differential Geometry of Curves and Surfaces*.

8.3 Complex analysis

Purpose

Complex analysis unifies local differentiability, global integral constraints, transforms, residues, harmonic functions, and conformal geometry. It is especially valuable in spectral methods, signal analysis, asymptotics, fluid and potential problems, and the evaluation of real integrals.

Core topics

- complex numbers, analytic functions, and Cauchy-Riemann equations;
- contour integration, Cauchy's theorem, and Cauchy's integral formula;
- power series, Laurent series, zeros, poles, and singularities;
- residues and the residue theorem;
- argument principle and Rouché-type reasoning;
- harmonic functions and potential theory;
- conformal maps;
- Fourier and Laplace transform connections;
- analytic continuation and branch structure.

Mastery gate

Produce a worked complex-analysis study that includes:

1. verification of analyticity on a stated domain;
2. a contour integral using Cauchy's theorem or residues;
3. a Laurent expansion and singularity classification;
4. a real integral or transform evaluated through complex methods;
5. an explicit account of branch cuts or domain restrictions;
6. a numerical contour check where appropriate.

Primary resource

MIT 18.04, Complex Variables with Applications, supplemented by Ahlfors or another rigorous complex-analysis reference.

9. Phase 6 - Functional analysis, numerical analysis, and optimisation

9.1 Functional analysis

Purpose

Functional analysis extends linear algebra from finite-dimensional vectors to spaces of functions, sequences, signals, and distributions. It provides the natural setting for differential equations, inverse problems, quantum models, kernels, variational methods, and operator spectra.

Core topics

- normed spaces, Banach spaces, and completeness;
- inner-product spaces and Hilbert spaces;
- bounded linear operators, continuity, adjoints, and operator norms;
- dual spaces, weak convergence, and weak-star ideas;
- Hahn-Banach, uniform boundedness, open mapping, and closed graph theorems;
- measure and integration sufficient for L^p spaces;
- orthonormal bases, Riesz representation, and projections;
- compact operators and Fredholm alternatives;
- self-adjoint operators and spectral theorems;
- unbounded operators, domains, and the need for additional care.

Mathematical cautions

- Infinite-dimensional spaces behave differently from finite-dimensional spaces: bounded sets need not be compact, norms need not be equivalent, and operators may be unbounded or lack eigenvectors.
- Compact operators map bounded sets to relatively compact sets. Approximation by finite-rank operators holds in standard Hilbert-space settings and in spaces with the relevant approximation property; it should not be stated without context.
- An unbounded operator is not defined on the whole ambient space in the same way as a bounded operator. Its domain is part of the object.

Mastery gate

Analyse one operator on a function space:

1. specify the function space, norm or inner product, and operator domain;
2. determine linearity and boundedness or explain unboundedness;
3. compute or characterise the adjoint where possible;
4. identify compactness, self-adjointness, or spectral properties under explicit assumptions;
5. construct a finite-dimensional approximation and compare it with the theoretical operator;
6. explain which conclusions would fail in a different norm or domain.

Primary resource

MIT 18.102, Introduction to Functional Analysis, supplemented by Kreyszig for engineering-oriented examples and more advanced references as required.

9.2 Numerical analysis

Purpose

Numerical analysis determines whether a mathematically valid method remains reliable when represented by finite-precision arithmetic and finite computational resources. It distinguishes a well-posed problem from a stable algorithm and a stable algorithm from a competent implementation.

Core topics

- floating-point arithmetic, machine precision, overflow, underflow, and cancellation;
- conditioning of problems versus stability of algorithms;
- forward error, backward error, residuals, and a posteriori checks;
- root finding and convergence rates;
- interpolation, polynomial approximation, splines, and Chebyshev methods;
- quadrature and integration error;
- direct and iterative methods for linear systems;
- QR, Cholesky, SVD, eigenvalue algorithms, and sparse matrices;
- Krylov methods such as conjugate gradients and GMRES;
- numerical ODEs, stiffness, adaptive methods, and stability regions;
- reproducibility, tolerance design, scaling, and benchmark construction.

Mastery gate

For one numerical problem, compare at least two algorithms and report:

1. mathematical problem and conditioning;
2. algorithmic assumptions;
3. convergence order or complexity;
4. residual, forward error, and backward error where available;
5. behaviour under scaling and perturbation;
6. floating-point failure cases;
7. a reproducible test harness and independent reference check.

Primary resources

MIT 18.330, Introduction to Numerical Analysis, followed by MIT 18.335J, Introduction to Numerical Methods. Trefethen and Bau is the preferred companion for numerical linear algebra.

9.3 Optimisation

Purpose

Optimisation converts objectives and constraints into decision procedures. The central structural distinction is between convex problems, where local information can often certify global conclusions, and non-convex problems, where local optima, saddle points, and problem geometry require greater caution.

Core topics

- convex sets, cones, convex functions, and epigraphs;
- first- and second-order optimality conditions;
- gradient, Newton, quasi-Newton, proximal, and coordinate methods;
- equality and inequality constraints;
- Lagrangians and Karush-Kuhn-Tucker conditions;
- weak and strong duality, constraint qualifications, and Slater's condition;
- linear, quadratic, conic, and semidefinite programming;
- regularisation and inverse problems;
- dynamic programming, Bellman equations, and optimal control;
- robust optimisation and min-max formulations;
- non-convex diagnostics, initialisation sensitivity, and local-versus-global claims.

Mathematical cautions

- KKT conditions are not automatically sufficient; sufficiency depends on convexity and appropriate constraint qualifications.
- A stationary point is not necessarily a local minimum.
- Strong duality requires conditions; it is not universal.
- Training loss, validation performance, calibration, and real-world utility are different objectives.
- Numerical termination criteria are evidence about an algorithm, not proof that a global optimum has been found.

Mastery gate

Formulate and solve one constrained optimisation problem:

1. define variables, objective, domain, and constraints;
2. establish convexity or identify the non-convex elements;
3. derive the Lagrangian and KKT conditions when applicable;
4. construct and interpret the dual problem;
5. solve numerically and verify primal feasibility, dual feasibility, and the duality gap;
6. perform sensitivity analysis;
7. document what the solver result does and does not certify.

Primary resource

Boyd and Vandenberghe, *Convex Optimization*, together with the associated Stanford course materials. Use a numerical optimisation course or solver documentation as a computational companion.

10. Synthesis practicum: translating mathematics into systems work

10.1 Purpose

The synthesis practicum prevents mathematics from becoming detached from actual design and prevents design language from becoming mathematically decorative. It runs through every phase.

For each important concept, maintain a translation record with seven fields:

Field	Question
Formal object	What is the object, its domain, and its notation?
Assumptions	What hypotheses are required?
Theorem or law	What result makes the object useful?
Operational meaning	What does it do geometrically, statistically, or dynamically?
System interpretation	Where does it appear in a model, algorithm, or architecture?
Computational test	What can be measured, simulated, or falsified?
Failure mode	What breaks when an assumption fails or the analogy is overextended?

10.2 Corrected example translations

Eigenvalues and linear stability

Formal object. For a linear map represented by A , a non-zero vector v is an eigenvector with eigenvalue λ when $A v = \lambda v$.

Law. For the finite-dimensional discrete linear system $x[k+1] = A x[k]$, the origin is asymptotically stable exactly when $\rho(A) < 1$, where ρ is the spectral radius.

Operational meaning. Eigenvalues describe modal growth, decay, oscillation, or neutral behaviour in an invariant direction or subspace.

System interpretation. They can characterise local modes of a linearised model, repeated iterations, filters, consensus processes, or state-transition operators.

Computational test. Compare predicted modal decay with direct simulation; inspect non-normal transient amplification and condition numbers of eigenvectors.

Failure mode. Local eigenvalues do not certify global nonlinear stability. Defective or non-normal matrices may behave poorly despite asymptotically stable eigenvalues.

Orthogonal projection

Formal object. An orthogonal projection P satisfies $P^2 = P$ and $P^* = P$.

Law. For a closed subspace V of a Hilbert space, $P_V x$ is the unique element of V minimising $\|x - v\|$.

Operational meaning. Projection selects the closest admissible representation under the chosen inner product.

System interpretation. Examples include least-squares fitting, reduced-order modelling, denoising, and enforcing linear constraints.

Computational test. Check idempotence, self-adjointness, residual orthogonality, and reconstruction error.

Failure mode. An oblique projection is idempotent but need not minimise Euclidean distance or preserve numerical conditioning.

Shannon entropy

Formal object. For a discrete random variable with probabilities $p(x)$, $H(X) = -\sum_x p(x) \log p(x)$.

Law. On a fixed finite support, entropy is maximised by the uniform distribution.

Operational meaning. Entropy is average uncertainty or expected code length under an optimal idealised code.

System interpretation. It can support compression analysis, uncertainty monitoring, active measurement, or routing under a probabilistic model.

Computational test. Estimate entropy under repeated samples, compare estimators, and assess finite-sample bias.

Failure mode. Entropy is not importance, complexity, truth, or causal relevance. Differential entropy also behaves differently from discrete entropy.

Lyapunov function

Formal object. A scalar function $V(x)$ defined near an equilibrium, usually with $V(0) = 0$ and $V(x) > 0$ for $x \neq 0$.

Law. Under standard regularity conditions, a positive-definite V with negative-definite derivative along trajectories gives local asymptotic stability. Global conclusions require stronger domain and growth conditions.

Operational meaning. V is an energy-like certificate that decreases along system trajectories.

System interpretation. It can certify controller stability, constrain adaptive updates, or establish a region of attraction.

Computational test. Evaluate V and its derivative or discrete difference across the claimed domain, including boundary and perturbation cases.

Failure mode. A positive score, loss, or distance is not a Lyapunov function unless its trajectory-wise decrease and domain conditions are proved.

Compact operator

Formal object. A linear operator is compact when it maps bounded sets to relatively compact sets.

Law. Compact operators on Hilbert spaces have strong approximation and spectral properties; compact self-adjoint operators admit an orthonormal eigenbasis for the closure of their range, with non-zero eigenvalues accumulating only at zero.

Operational meaning. Compactness is one route by which an infinite-dimensional problem can display controlled finite-dimensional approximability.

System interpretation. It appears in integral equations, smoothing operators, kernel methods, and inverse problems.

Computational test. Study singular-value decay and convergence of finite-rank or discretised approximations.

Failure mode. Non-compact or unbounded operators do not share the same spectral and approximation behaviour. Discretisation can conceal this distinction.

Curvature

Formal object. For a unit-speed curve with unit tangent $T(s)$, curvature is $\kappa = ||dT/ds||$. For a surface, Gaussian curvature is the product of the principal curvatures.

Law. Curvature is invariant under suitable reparametrisation and links local metric behaviour to geometric structure.

Operational meaning. It measures local bending or departure from flatness.

System interpretation. Curvature can describe constrained state spaces, parameter manifolds, trajectory bending, or local optimisation geometry.

Computational test. Compare analytic curvature with finite-difference or automatic-differentiation estimates under coordinate changes.

Failure mode. Local curvature does not establish global convexity, topology, or reachability. A poorly scaled coordinate system can distort naive numerical estimates.

10.3 Monthly mathematical artifact

Produce at least one substantial artifact per month:

- proof memo;
- derivation memo;
- theorem-to-system translation;
- numerical experiment with error analysis;
- counterexample notebook;
- reproduction of a published result;
- “where the analogy breaks” memo;
- formal claim sheet with assumptions, evidence, and unresolved obligations.

A useful artifact is self-contained. Another reader should be able to identify the claim, inspect the method, reproduce the computation, and see the unresolved limitations.

10.4 Where the analogy breaks

Analogy is valuable for discovery and dangerous as evidence. For any cross-domain mapping, record:

1. which objects correspond;
2. which operations are preserved;
3. which invariants are preserved;
4. whether the mapping is injective, surjective, invertible, approximate, or merely heuristic;
5. the first property that fails;
6. the experiment or counterexample that exposes the failure.

This distinction prevents a visual resemblance or shared vocabulary from being mistaken for a theorem.

11. Study cadence and evidence of mastery

11.1 Four-session weekly cycle

Session	Function	Typical output
A - Theory	Read definitions, theorem statements, and worked examples	Definitions ledger and dependency notes
B - Proof	Solve proof and derivation problems without immediately consulting solutions	Completed proof, failed-attempt record, corrected proof
C - Computation	Implement a small numerical experiment	Reproducible notebook, plots, residuals, and error discussion
D - Translation	Connect the mathematics to a system, model, or research claim	Translation record or claim sheet

The sessions need not be equal in length. The important requirement is that none of the four functions disappears for long periods.

11.2 Three-pass learning method

Pass 1 - Intuition

Ask:

- What objects exist in this theory?
- What kinds of problems does the theory solve?
- What is the geometric, probabilistic, or dynamical picture?
- What simple examples and counterexamples expose the main distinctions?

Pass 2 - Formal layer

Do:

- write definitions exactly;
- state theorems with all hypotheses;
- understand at least one proof of each central result;
- solve problems that require selecting the theorem rather than imitating an example;
- identify assumptions that are easy to overlook.

Pass 3 - Engineering layer

Ask:

- How is the object represented in computation?
- What can be measured or tested?

- How does numerical error alter the conclusion?
- What breaks when the theorem's assumptions fail?
- What claim about a real system can now be made more precisely?

11.3 Use of solutions and computational tools

A solution should be consulted only after a genuine attempt has produced a specific blockage. Record the blockage first. Then close the solution and reproduce the argument independently.

Computer algebra, numerical libraries, theorem provers, and language models can support study, but they do not replace the obligation to:

- define the problem correctly;
- inspect assumptions;
- verify dimensions and domains;
- check numerical conditioning;
- test edge cases;
- reproduce critical steps without opaque dependence on a tool.

11.4 Evidence ladder

Use the following evidence ladder for technical claims:

1. **Intuitive:** plausible explanation or analogy.
2. **Worked example:** demonstrated on selected cases.
3. **Computational evidence:** reproducible experiments over a stated test domain.
4. **Derivation:** result follows algebraically from declared assumptions.
5. **Theorem-backed:** a named theorem applies and every hypothesis has been checked.
6. **Independent verification:** a second method, implementation, reviewer, or source supports the result.

Higher levels do not erase lower-level concerns. A theorem about an idealised model does not validate an incorrect implementation, and successful experiments do not prove a universal claim.

11.5 Review interval

At the end of each month:

- select one proof for external or delayed self-review;
- rerun one numerical experiment from a clean environment;
- revisit one failed argument;
- update the definitions ledger;
- identify one concept whose analogy has been overextended;
- decide whether the current phase gate is genuinely passed.

12. Suggested pathways and emphasis order

The curriculum is competency-based, so no universal duration is specified. The following pathways prioritise topics without changing their prerequisites.

12.1 General systems route

1. Phase 0 - proof and discrete reasoning
2. Phase 1 - linear algebra
3. Phase 2 - real analysis
4. Phase 3 - probability and information
5. Phase 4 - dynamics and control
6. Phase 6b - numerical analysis
7. Phase 6c - optimisation
8. Phase 5 and Phase 6a according to project needs

12.2 AI and machine-learning route

1. Phase 0 diagnostic
2. Phase 1, with emphasis on SVD, conditioning, quadratic forms, and state-space models
3. Phase 3, with emphasis on inference, KL divergence, mutual information, and calibration
4. Phase 2, with emphasis on convergence and fixed points
5. Phase 6b and 6c, with emphasis on numerical linear algebra and optimisation
6. Phase 4 for iterative dynamics, stability, feedback, and distribution shift over time
7. Functional analysis or geometry when model structure requires it

12.3 Control and robotics route

1. Phase 0 diagnostic
2. Phase 1, including controllability and observability
3. Phase 2, especially fixed points, compactness, and uniform convergence
4. Phase 4 in depth
5. Phase 6b and 6c
6. Probability and stochastic processes
7. Differential geometry for constrained and manifold-valued systems

12.4 Mathematical research route

Follow the complete sequence. Add a proof-based multivariable calculus or topology course before advanced geometry, and add measure theory before deeper functional analysis or probability. The synthesis practicum should include theorem reconstruction, counterexamples, and computational falsification.

12.5 Priority rationale

Priority	Subject	Reason
1	Proof and discrete mathematics	Prevents ambiguity from propagating into every later phase
2	Linear algebra	Highest general return for representation, algorithms, spectra, and state models
3	Real analysis	Controls convergence, approximation, limits, and theorem hypotheses
4	Probability and information	Formalises uncertainty, inference, dependence, and evidence
5	Dynamics and control	Formalises evolution, stability, intervention, and feedback
6	Numerical analysis	Ensures methods survive finite precision and discretisation
7	Optimisation	Converts objectives and constraints into decision procedures
8	Algebra, geometry, complex analysis	Adds structural depth according to domain needs
9	Functional analysis	Provides the infinite-dimensional operator framework once analysis is secure

13. Mastery checkpoints

Each checkpoint requires a written, reviewable artifact.

Checkpoint	Demonstrated capability
CP-1	Write valid proofs, manipulate quantifiers, and establish invariants without ambiguity
CP-2	Characterise linear maps using subspaces, projections, SVD, spectra, conditioning, and state-space behaviour
CP-3	State and prove convergence claims with the correct mode, space, and hypotheses
CP-4	Model uncertainty and compute inferential and information-theoretic quantities with correct interpretation
CP-5	Analyse a dynamical system, distinguish local from global claims, and design or assess feedback
CP-6	Identify whether a new problem is principally algebraic, geometric, analytic, probabilistic, dynamical, numerical, or optimisation-based
CP-7	Translate a system claim into formal objects, assumptions, invariants, observables, computations, and failure modes
CP-8	Produce a reproducible technical artifact whose conclusions are no stronger than its evidence

13.1 Terminal checkpoint

The terminal checkpoint is not memorisation of the largest number of theorems. It is the ability to take a consequential technical claim and produce a claim package that an informed sceptic can inspect end-to-end:

- exact statement;
- domain and objects;
- assumptions;
- relevant theorem or derivation;
- implementation or computational method;
- verification plan;
- counterexamples and boundary conditions;
- residual uncertainty;
- evidence classification;
- reproducible artifact.

14. Resource policy

14.1 Use one primary spine

For each phase, choose one primary course and complete its sequence, exercises, and assessments. Add a companion only when it fills a specific gap. Continually switching sources creates the illusion of progress while avoiding the difficult work of consolidation.

14.2 Resource tiers

Tier	Function	Examples
P1 - Primary	Ordered instruction with definitions, problems, and assessments	MIT OpenCourseWare, Stanford EE263, Underactuated Robotics, Boyd and Vandenberghe
C2 - Companion	Alternative explanation or additional exercises	OpenStax, Tao, Strogatz, Trefethen and Bau
R3 - Reference	Dense lookup or advanced treatment	Rudin, Ahlfors, Kreyszig, Reed and Simon
D4 - Discovery	Find specialist material after the primary spine is stable	Curated bibliographies, arXiv, nLab, author lecture notes

14.3 Source evaluation checklist

Before adopting a resource, check:

- author, institutional provenance, level, and prerequisites;
- whether it includes exercises, solutions, and clearly stated theorem hypotheses;
- publication date, revision history, and available errata;
- licensing terms and whether the source is primary instruction, reference, or informal commentary.

14.4 Papers and advanced notes

Research papers are not substitutes for prerequisite courses. Use them after the relevant formal language is stable. For each paper, record the claim, assumptions, proof or empirical evidence, dependence on cited results, implementation details, reproducibility status, and unresolved questions.

15. Resource directory

The following links were checked for this public edition on 5 August 2026. External sites may later reorganise or replace materials.

15.1 Primary online courses and notes

Area	Resource	Link
Proof and discrete mathematics	MIT 6.042J - Mathematics for Computer Science	Official course page
Linear algebra	MIT 18.06SC - Linear Algebra	Official course page
Applied linear systems	Stanford EE263 - Introduction to Linear Dynamical Systems	Official course page
Matrix calculus	MIT 18.S096 - Matrix Calculus for Machine Learning and Beyond	Official course page
Real analysis	MIT 18.100B - Real Analysis	Official course page
Probability	MIT 6.041SC - Probabilistic Systems Analysis and Applied Probability	Official course page
Information theory	MIT 6.441 - Information Theory	Official course page
Differential equations	MIT 18.03 - Differential Equations	Official course page
Nonlinear control	MIT Underactuated Robotics	Official course notes
Abstract algebra	MIT 18.701 - Algebra I	Official course page
Differential geometry	MIT 18.950 - Differential Geometry	Official course page
Complex analysis	MIT 18.04 - Complex Variables with Applications	Official course page
Functional analysis	MIT 18.102 - Introduction to Functional Analysis	Official course page
Numerical analysis	MIT 18.330 - Introduction to Numerical Analysis	Official course page
Numerical methods	MIT 18.335J - Introduction to Numerical Methods	Official course page
Convex optimisation	Boyd and Vandenberghe - Convex Optimization	Official textbook page

15.2 Open textbooks and companion material

Resource	Use	Link
OpenStax Calculus, Volume 1	Calculus repair and reference	Open textbook
OpenStax Introductory Statistics 2e	Practical statistics reference	Open textbook
Sheldon Axler, Linear Algebra Done Right	Coordinate-free linear algebra companion	Author page
Terence Tao, Analysis books and notes	Analysis companion and further study	Author page
Boyd and Vandenberghe, Convex Optimization PDF	Optimisation textbook	Open PDF

15.3 Standard references

- Walter Rudin, *Principles of Mathematical Analysis*.
- Michael Artin, *Algebra*.
- Roger Horn and Charles Johnson, *Matrix Analysis*.
- Steven Strogatz, *Nonlinear Dynamics and Chaos*.
- Thomas Cover and Joy Thomas, *Elements of Information Theory*.
- Erwin Kreyszig, *Introductory Functional Analysis with Applications*.
- Manfredo do Carmo, *Differential Geometry of Curves and Surfaces*.
- Lars Ahlfors, *Complex Analysis*.
- Lloyd Trefethen and David Bau, *Numerical Linear Algebra*.
- Michael Reed and Barry Simon, *Methods of Modern Mathematical Physics, Volume I: Functional Analysis*.

15.4 Advanced discovery sources

Use only after the relevant foundation is stable:

- arXiv subject archives for mathematics, statistics, control, and computation;
- nLab for category-theoretic and structural terminology;
- author-maintained lecture notes with clear provenance and revision history;
- review articles and textbooks that synthesise a field before individual research papers.

16. Working templates

16.1 Translation record

Concept:
 Formal object and domain:
 Assumptions:
 Theorem or law:
 Operational meaning:
 System interpretation:
 Computational test:
 Failure mode:
 Open questions:

16.2 Proof memo

Claim:
 Definitions:
 Assumptions:
 Method:
 Proof:
 Dependencies:
 Boundary cases:
 Counterexample if an assumption is removed:
 Reviewer notes:

16.3 Numerical experiment record

Question:
 Mathematical model:
 Algorithm:
 Implementation and environment:
 Reference solution or invariant:
 Parameters and seeds:
 Residual and error metrics:
 Conditioning and stability concerns:
 Results:
 Failure cases:
 Reproduction command:

16.4 Claim sheet

Claim identifier:
 Claim statement:
 Scope:
 Evidence level:
 Assumptions checked:
 Theorem or derivation:
 Empirical verification:
 Independent check:
 Known limitations:
 Next falsification test:

16.5 Where the analogy breaks memo

Source domain:
 Target domain:
 Proposed correspondence:
 Preserved objects:
 Preserved operations:
 Preserved invariants:
 Type of mapping:
 First failed property:
 Counterexample or test:
 Permitted conclusion:
 Conclusion that is not justified:

17. Final operating principle

Build technical mathematics around proof, operators, uncertainty, dynamics, and computation. Maintain a live translation layer between formal results and real systems.

Mathematical depth is not useful in isolation, and engineering intuition is not trustworthy without explicit assumptions and evidence. The objective is to keep both sides connected:

- **Rigor:** know whether a claim is proved, derived, observed, or merely plausible.
- **Expressive power:** state ideas precisely enough to be tested.
- **Falsification:** identify the assumption or experiment that could disprove the claim.
- **Compression:** reduce a complex system to the smallest set of objects, laws, invariants, and observables that still explains its behaviour.
- **Reliability:** ensure the computational implementation preserves the mathematical meaning closely enough for the conclusion to remain valid.

The curriculum is complete only when a reader can move in both directions: from a system to its mathematical structure, and from a theorem back to a measurable, reproducible system claim.

Mathematics for Systems Research and Engineering - Public Edition 2026

Copyright (c) 2026 Thomas Helm / ToraFirma Systems. All rights reserved.

Repository: [ToraFirma Systems public repository](#)